

23CSE211 - DESIGN AND ANALYSIS OF ALGORITHMS

Attempts on solving Flow-Free - 2 : Electric Boogaloo

Greedy, Divide and Conquer

Submitted by:

Akilan S S	–	CB.SC.U4CSE24707
Govind Nair	–	CB.SC.U4CSE24720
M V Sai Kartik	–	CB.SC.U4CSE24744
Sriram Tatikonda	–	CB.SC.U4CSE24753

Submitted to:

Dr. Vidhya Balasubramanian

Wednesday 18th February, 2026

Contents

1	Introduction	2
2	Radial Weighted Greedy	2
2.1	Description	2
2.2	Time Complexity Analysis	2
3	Naive Divide and Conquer	4
3.1	Description	4
3.2	Time Complexity Analysis	4
4	Uneven region with path finding algorithm	6
4.1	Description	6
4.2	Time Complexity Analysis	6
4.3	How I got to this algorithm	8
5	BFS with Bending through Region Finding	10
5.1	Description	10
5.2	Time Complexity Analysis	10
5.3	Observations and feasibility	12
6	Dynamic Uneven Region Segmentation Algorithm	13
6.1	Description	13
6.2	Example	13
6.3	Time Complexity Analysis	16
7	Conclusion and Future Directions	18
8	LLM Disclosure	19

1 Introduction

Flow-free is a grid based game, where in an n by n grid, there are k pairs of colours (or terminals), which has to be connected by drawing pipes on the grid. No two pipes must intersect, and they should fill the grid as much as possible. In this project, we attempt to solve the game Flow-Free with traditional algorithm techniques, presented till date in 23CSE211, in a collaborative manner through the following strategies. These strategies are placed in order of nonchalantness. Lower the number lower the nonchalantness.

1. Radial Weighted Greedy
2. Naive Divide and Conquer
3. Recursive Bending
4. Divide and Conquer with Region Finding
5. Uneven Region Segmentation

2 Radial Weighted Greedy

2.1 Description

This strategy aims to force the paths taken to wrap around the puzzle, in a essence it takes the longer path to connect terminals. This algorithm although overcomes difficulties faced in the previous labs, it has it's own share of problems. We also need to order the connection of terminals based on the distance between the pair. This ensures that the djikstra doesn't run over and cover all escaping cells of a terminal.

TO make the radial-ness actually work, every ring of the grid are weighted exponentially. Exponentially weighting forces the scenario that to travel a cell in a ring i is equivalent to travelling 10^i cells in ring $i + 1$. A more optimized weighting policy might include the size of grid as well, but this works fine.

2.2 Time Complexity Analysis

For a puzzle with $N \times N$ grid, with K terminals

- **Sorting terminals based on distance:** $\mathcal{O}(K \log K)$
- **Weighting cells:** $\mathcal{O}(N^2)$
- **Dijkstra:** $\mathcal{O}(N^2)$

Algorithm 1 Greedy Wrap Algorithm

```
procedure GREEDYWRAP(board)  
    terminals  $\leftarrow$  IDENTIFYTERMINALS(board)  $\triangleright O(n^2)$   
    pairs  $\leftarrow$  []  
    for each color  $\in$  terminals do  
        dist  $\leftarrow$  EUCLIDEANDISTANCE(p1, p2)  
        pairs.push({dist, color, p1, p2})  
    end for  
    SORT(pairs)  $\triangleright$  Shortest pairs first  
    for each p  $\in$  pairs do  
        path  $\leftarrow$  DIJKSTRA(board, p.start, p.end)  
        if path is not  $\emptyset$  then  
            return path  $\triangleright$  Returns the first valid path found  
        end if  
    end for  
    return  $\emptyset$   
end procedure  
function DIJKSTRA(board, start, end)  
    PQ.push({0, start})  
    while PQ not empty do  
        curr  $\leftarrow$  PQ.pop()  
        for each neighbor  $\in$  curr do  
            w  $\leftarrow$  10distToBorder(neighbor)  
            if neighbor is free or target then  
                RELAXEDGE(curr, neighbor, w)  
            end if  
        end for  
    end while  
end function
```

3 Naive Divide and Conquer

3.1 Description

This sound exactly like what one would very obviously think of. The nature of the game flow free is that, it can be extended to an arbitrarily large grid. The terminals of these grids can also be locally connected. What this means is, if you take a grid of size dimension $N \times M$ with K , there could exist a subgrid $m \times n$ with k terminals, which also forms a locally small flow-free game.

With this idea in mind, one way to implement an divide and conquer algorithm is to divide the grid into 4 quarters (since it is a 2d grid), and solve them recursively. One challenging part of this is, not every subgrid form a locally small flow-free game. Naively dividing the grid into quarters may lead some terminals stuck across different subgrid, which should be taken care of.

If you divide into quarters, at the worst case you have to find some way to carry over a terminal's path across 3 subgrid which becomes super messy even think of. To save our sanity, we decided bisecting the grids instead. But this bisection will happen in different directions alternatively to preserve the 'squareness' of the subgrids.

3.2 Time Complexity Analysis

Let n be the number of rows (or columns) in the grid (where $N = n^2$ total cells), and k be the number of available color terminal pairs.

- **Scan for terminals:** $O(n^2)$
Time required to scan the board, identify terminal locations, and populate the `terminalMap`.
- **Recursive Splitting:** $O(\log n)$
The algorithm employs a divide-and-conquer strategy, splitting the grid dimensions in half at each step. This results in a recursion tree depth of logarithmic order relative to the grid size.
- **Pathfinding Execution (Dijkstra):** $O(k \cdot n^2 \log n)$
In each recursive call, the algorithm identifies k potential straddling colors. For each valid color, Dijkstra's algorithm is executed. In the worst case, Dijkstra visits every vertex and edge in the current grid segment (n^2).

Recurrence Relation:

The complexity of the recursive solver can be expressed as:

$$T(N) = 2T\left(\frac{N}{2}\right) + f(k, N)$$

Where N is the number of cells and $f(k, N) = O(k \cdot N \log N)$ represents the work of finding paths for k colors using a priority-queue-based Dijkstra.

Total Time Complexity: $O(k \cdot n^2 \log n)$

Algorithm 2 Half-Split Recursive Pathfinding

```
procedure HALFSPLITSOLVER(board, topLeft, bottomRight)
  if topLeft = bottomRight or invalid(bounds) then
    return
  end if
  (H1, H2) ← FINDHALF(topLeft, bottomRight)           ▷ Split by larger dimension
  straddlers ← []
  for each color ∈ terminalMap do
    t1, t2 ← terminalMap[color]
    if IsStraddling(t1, t2, H1, H2) and notVisited(t1) then
      dist ← GETDISTANCE(t1, t2)
      straddlers.push(dist, color)
    end if
  end for
  SORTBYDISTANCE(straddlers)                               ▷ Heuristic priority
  for each (d, color) ∈ straddlers do
    path ← DIJKSTRA(t1, t2, {topLeft, bottomRight})
    if path ≠ ∅ then
      MARKVISITED(path)
      validPaths.add(path)
    end if
  end for
  HALFSPLITSOLVER(board, H1)                             ▷ Recurse into first half
  HALFSPLITSOLVER(board, H2)                             ▷ Recurse into second half
end procedure
```

4 Uneven region with path finding algorithm

4.1 Description

The algorithm groups terminals by color, prioritizes closer pairs, builds a geometric region between each terminal pair to constrain movement, and then uses DFS to find a valid grid path between terminals using only cells inside that region.

4.2 Time Complexity Analysis

- Let the board be $N \times M$
 - Let K be the number of colors (terminal pairs)
 - Let $C = N \cdot M$ be the total number of cells
 - Let R be the number of cells where $\text{region}[i][j] = 1$
1. **getTerminals()** — $O(NM)$
Getting the Location of terminals with their colours, with colour as key and value as the location of terminals
 2. **getDistanceColor()** — $O(K \log K)$
Getting the distances of colours in non-decreasing order with distance as key and colours as value.
 3. **getRegion()** — $O(NM)$
Getting the matrix of region with 1 as value for the cells that are inside the region, else 0.
 4. **dfs()** — $O(NM)$
Running DFS to find path inside the region.
 5. Total work per color — $O(NM)$
 6. Worst case (computing for every color) — $O(KNM)$

Total Time Complexity: $O(KNM)$

Algorithm 3: Uneven region with DFS algorithm

```
1: function SOLVEBOARD(board)
2:   terminals  $\leftarrow$  GETTERMINALS(board)
3:   distColors  $\leftarrow$  GETDISTANCECOLOR(terminals)
4:   for all distance in distColors (ascending) do
5:     for all color in distColors[distance] do
6:        $(s, e) \leftarrow$  terminals[color]
7:       region  $\leftarrow$  GETREGION(s,e,board)
8:       path  $\leftarrow$  DFSFINDPATH(s,e,board,region)
9:       if path  $\neq \emptyset$  then
10:        return path
11:    end if
```

```

12:     end for
13: end for
14: return  $\emptyset$ 
15: end function

16: function GETTERMINALS(board)
17:     terminals  $\leftarrow$  empty map
18:     for all cells  $(i, j)$  in board do
19:         if cell is terminal AND has no pipe then
20:             if color not in terminals then
21:                 terminals[color].start  $\leftarrow (i, j)$ 
22:             else
23:                 terminals[color].end  $\leftarrow (i, j)$ 
24:             end if
25:         end if
26:     end for
27:     return terminals
28: end function

29: function GETDISTANCECOLOR(terminals)
30:     distColors  $\leftarrow$  empty ordered map
31:     for all (color, (t1,t2)) in terminals do
32:         if t2 is valid then
33:              $d \leftarrow |t1.x - t2.x| + |t1.y - t2.y|$ 
34:             Append color to distColors[d]
35:         end if
36:     end for
37:     return distColors
38: end function

39: function GETREGION(start,end,board)
40:     Initialize region matrix to 0
41:     Compute center of line (start, end)
42:     Compute perpendicular direction
43:     Form polygon using terminals and perpendicular endpoints
44:     for all cells  $(i, j)$  in board do
45:         if POINTINPOLYGON(polygon, sample points of  $(i, j)$ ) then
46:             region[i][j]  $\leftarrow$  1
47:         end if
48:     end for
49:     return region
50: end function

51: function POINTINPOLYGON(polygon, point)
52:     inside  $\leftarrow$  FALSE
53:     for all edges  $(v_i, v_j)$  in polygon do

```



```

54:         if ray from point intersects edge then
55:             inside  $\leftarrow \neg inside$ 
56:         end if
57:     end for
58:     return inside
59: end function

60: function DFSFINDPATH(start,end,board,region)
61:     visited  $\leftarrow$  FALSE matrix
62:     path  $\leftarrow \emptyset$ 
63:     if DFSUTIL(start,start,end,board,region,visited,path) then
64:         return path
65:     end if
66:     return  $\emptyset$ 
67: end function

68: function DFSUTIL(i,j,end,board,region,visited,path)
69:     if out of bounds OR region[i][j]=0 OR visited[i][j] OR cell blocked then
70:         return FALSE
71:     end if
72:     visited[i][j]  $\leftarrow$  TRUE
73:     Add (i,j) to path
74:     if (i,j) = end then
75:         return TRUE
76:     end if
77:     for all 4-neighbors (ni,nj) of (i,j) do
78:         if DFSUTIL(ni,nj,end,board,region,visited,path) then
79:             return TRUE
80:         end if
81:     end for
82:     Remove (i,j) from path
83:     return FALSE
84: end function

```

4.3 How I got to this algorithm

First I thought of using a region finding algorithm that tries to maximize some sort of score and run some type of path finding algorithm to find the solution. I thought, I will label the terminals I want to connect as 2 and empty spaces as 1 and everything as -1.

But then I realise, this divide and conquer algorithm does nothing because if I find the region that is connected by empty cells and has the terminals that I want to connect in it, and run some pathing finding algorithm, it does the same thing as running the path finding algorithm normally. **So, this type of region finding algorithm is basically pretending to do something but does nothing. I thought I will label this algorithm as politician of all algorithms.**

Then I thought of Three phase algorithm. Where I will start with a single cell in the center of the board.

Phase one:

I will choose one terminal of the two terminals I want to connect, and I will add cells to my region that goes towards the chosen terminal. I will do this until I reach the terminal or added the terminal in my region.

Phase Two:

Next I will repeat the same process from the terminal in my region to connect the other terminal that is not in the region.

Phase Three:

I will calculate distance for each cell to both the terminals and add them and if the distance is above a certain threshold, I will remove that cell from the region.

I realised that if I run this algorithm, I will get some sort of triangle in some cases. This gave me the idea to take the region as square or rhombus with the terminals as two edge points.

5 BFS with Bending through Region Finding

5.1 Description

In this strategy, we build on top of the sorting + BFS strategy in the previous attempt and try to enhance it by using a flood fill and region-finding algorithm to fill neighbouring empty cells. In practice, this should work for sparser graphs as there is a lot more space to be filled. This involves the following three main functions:

findEmptyRegions: This function divides the grid into different regions of groups of empty cells. It does this by scanning the grid cell by cell and running a flood fill from a cell if it is undiscovered until it hits a terminal or existing pipe. This is now one region. This continues until all such regions are found.

regionPath: This function builds on the bfsPath function from Attempt 1 but instead runs a modified Dijkstra's algorithm to find the path between the two terminals where a cell belonging to a larger region has a lower cost.

algorithm2: This is the main algorithm which invokes regionPath and findEmptyRegions. Instead of sorting by shortest distance between terminals, we do the opposite to minimise the chance of blocking other potential paths. It then rescores the pairs by how large the region is between the two terminals and runs regionPath on the most optimal pair.

5.2 Time Complexity Analysis

- **findEmptyRegions:** $O(n^2)$
We go through every cell of the grid to check for validity.
- **regionPath:** $O(n^2 \log n)$
This function is dominated by the modified Dijkstra's algorithm. Since we use a priority queue in the algorithm, each cell can enter the queue up to $4n^2$ times while the cost of pushing/popping from the queue takes $O(\log n)$ time making the total complexity $O(n^2 \log n)$.
- **algorithm2:** $O(P \cdot n^2 \log n)$
The algorithm runs regionPath for P pairs.

Algorithm 4: findEmptyRegions(board)

```
1: mark all cells as unvisited
2: regions ← empty list
3: for each cell  $(r, c)$  in board do
4:   if cell is empty AND not visited then
5:     region ← all empty cells connected to  $(r, c)$ 
6:     add region to regions
7:   end if
8: end for
9: return regions
10: return regions
```

Algorithm 5: regionPath(board, start, goal)

```
shortest  $\leftarrow$  shortest valid path from start to goal
if shortest does not exist then
    return failure
end if
limit  $\leftarrow$  length(shortest) + allowedExtra
regions  $\leftarrow$  FINDEMPYREGIONS(board)
assign each cell a value based on size of its region
Perform priority-based search from start where:
    shorter paths preferred
    larger regions preferred
    paths beyond limit ignored
    blocked cells skipped
if goal reached then
    path  $\leftarrow$  reconstructed search path
    if length(path) within limit then
        return path
    end if
end if
return shortest
```

Algorithm 6: algorithm2(board)

```
1: collect terminals grouped by color
2: build list of terminal pairs
3: remove invalid or already solved pairs
4: order pairs so longer distances first
5: regions  $\leftarrow$  FINDEMPYREGIONS(board)
6: for each pair do
7:     compute score based on distance
8:     increase priority if large regions lie near both terminals
9: end for
10: sort pairs by priority
11: for each pair in order do
12:     attempt  $\leftarrow$  REGIONPATH(board, pair.start, pair.end)
13:     if attempt found then
14:         return attempt
15:     end if
16: end for
17: return failure
```

5.3 Observations and feasibility

I ran this algorithm on multiple game boards. What did I observe?

- **Dense Graphs:** This algorithm works on dense graphs ONLY IF A SPECIFIC TERMINAL ORDER IS FOLLOWED. In cases where a region is shared by multiple terminal pairs, there is a very high chance that the new bent path will cover the entire region and block potential new paths.
- **Sparse Graphs:** This graph does a better job in cell filling than regular sorting + BFS but shares the same constraint of the BFS choosing a path that can potentially block other paths.

So in hindsight, the algorithm doesn't really do a good job. Is there any way I can save it by giving it a feasible use case?

After further testing, this algorithm works pretty well in sparse graphs, especially those which have a smaller number of terminal pairs (for obvious reasons, since one single region exists which covers the remaining grid space whose path blows that of regular BFS out of the park).

A good use case of this would be to pair this algorithm with any other algorithm (like BFS path finding) and to run in specifically when the number of remaining terminals to be paired is low. In situations where the final terminal pairs require a lot of path wrapping, this can come in extremely handy in filling the grid instead of leaving out empty spaces.

So, while this is not a massive improvement compared to regular sorting + BFS, it now considers the objective of space filling as well.

6 Dynamic Uneven Region Segmentation Algorithm

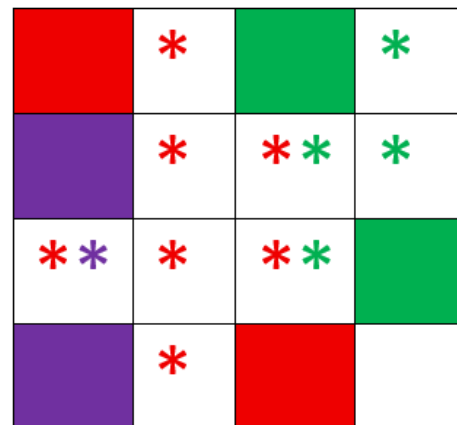
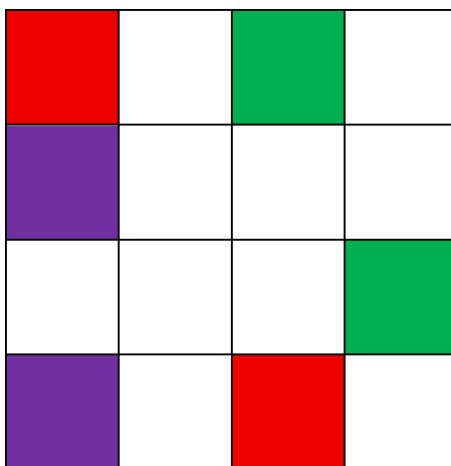
6.1 Description

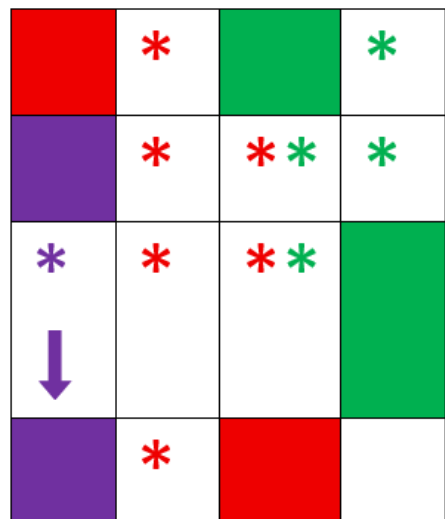
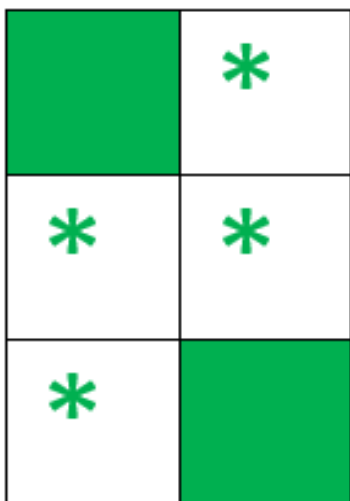
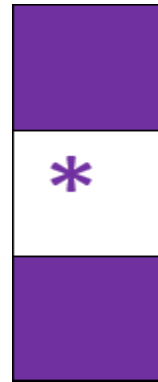
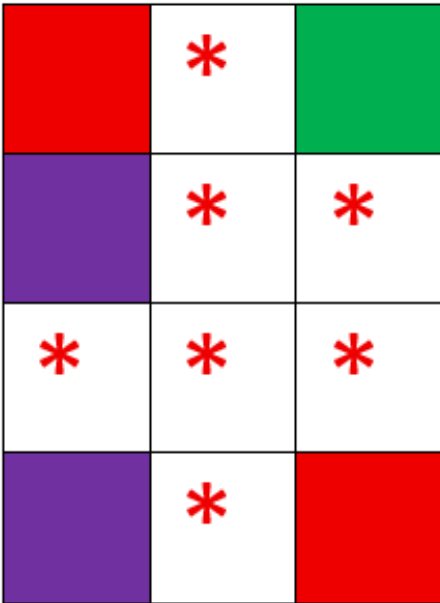
The Dynamic Uneven Region Segmentation Algorithm solves the Flow puzzle by dynamically restricting the search space of each color pair. For every unfinished color, it constructs the smallest bounding rectangle containing its terminals and attempts to find a path within this constrained region. If no path exists, the region is expanded incrementally while maintaining global consistency.

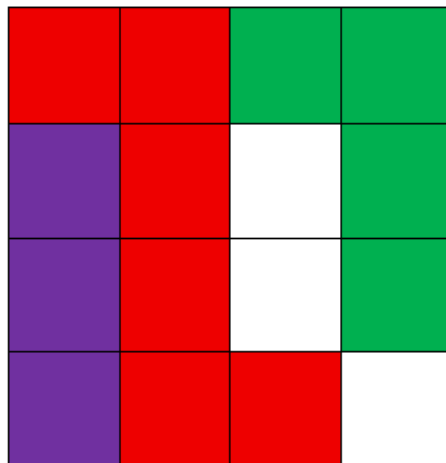
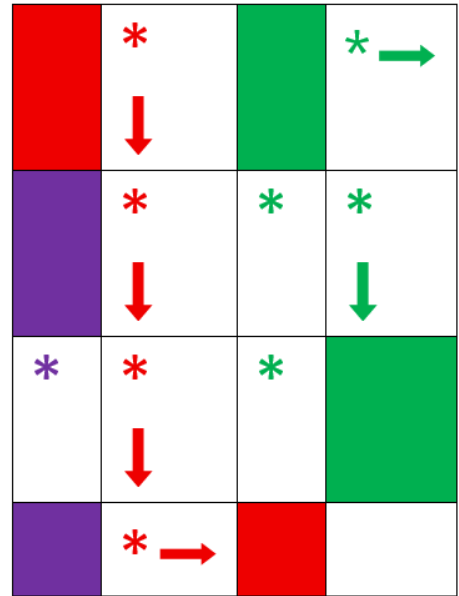
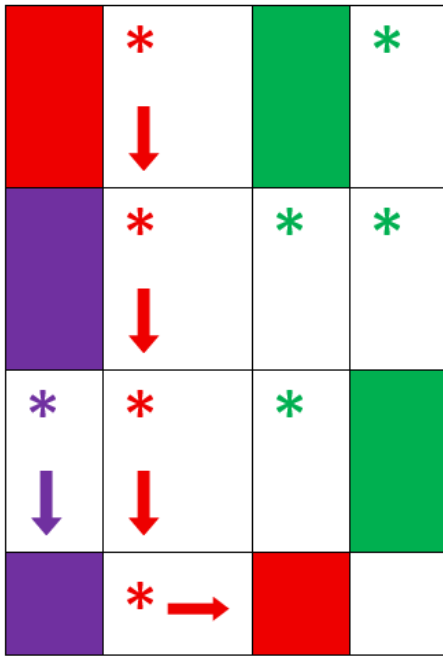
The approach follows a Divide and Conquer strategy: the board is divided into localized rectangular regions (divide phase), and candidate paths from these regions are evaluated and reconciled globally to select the least conflicting move (combine phase). By iteratively committing one valid path at a time, the algorithm reduces complexity without resorting to brute force or full backtracking.

6.2 Example

Board Evolution







Example Illustration

6.3 Time Complexity Analysis

buildRegion (rectangle + optional flood fill): $\mathcal{O}(N^2)$ We scan or expand over the grid region for a color. In the worst case, the region can cover the entire board.

bfsShortestPath: $\mathcal{O}(N^2 \log N)$ This step is dominated by the priority-queue based shortest path search.

For a grid region of size $\leq N^2$:

$$V \leq N^2$$

$$E = \mathcal{O}(N^2)$$

Using a priority queue gives:

$$\mathcal{O}(V \log V) = \mathcal{O}(N^2 \log N)$$

perColorWork: $\mathcal{O}(N^2 \log N)$ Each unfinished color performs region building plus path search. The path search dominates, so total per color is $\mathcal{O}(N^2 \log N)$.

solve (single call): $\mathcal{O}(KN^2 \log N)$ We repeat perColorWork for K colors. Forward checking also repeats similar work, but remains within the same asymptotic bound.

Full puzzle (worst case): $\mathcal{O}(K^2 N^2 \log N)$ If up to K paths are fixed sequentially (one per color), we may call solve up to K times.

Final Result

$$\text{Single solve step} \rightarrow \mathcal{O}(KN^2 \log N)$$

$$\text{Complete puzzle (worst case)} \rightarrow \mathcal{O}(K^2 N^2 \log N)$$

Algorithm 7 Dynamic Uneven Region Segmentation algorithm

```
1: function SOLVEONEMOVE( $B$ )
2:   Ensure terminal pairs exist in  $B$ 
3:    $R \leftarrow$  minimal rectangles for each color pair
4:    $Candidates \leftarrow \emptyset$ 
5:   for each unfinished color  $c$  do
6:      $S \leftarrow R[c]$ 
7:     Remove blocked / foreign cells from  $S$ 
8:      $P \leftarrow$  shortest path inside  $S$ 
9:     if  $P$  empty then
10:      Expand  $S$  via flood-fill
11:       $P \leftarrow$  shortest path inside expanded  $S$ 
12:     end if
13:     if  $P$  exists then
14:       Store  $(c, P, heuristic.info)$  in  $Candidates$ 
15:     end if
16:   end for
17:   if  $Candidates$  empty then
18:     return false
19:   end if
20:   Sort  $Candidates$  by:
   fewest paths, smallest rectangle, largest terminal distance
21:   Let  $c^*$  be best-ranked color
22:    $bestPath \leftarrow$  path minimizing:
   (1) conflict with other pipes
   (2) path length
   and passing forward-check
23:   if  $bestPath$  not found then
24:     return false
25:   end if
26:   Lock  $bestPath$  on board
27:   return true
28: end function
```

7 Conclusion and Future Directions

In this phase we explored (smashed our head against the wall) different ways to incorporate yet another greedy strategy, and come up with a divide and conquer strategy. As expected it had it's ups and downs. In the next and final phase of lab evaluation of this course, we shall deal the final blow by figuring out dynamic programming and backtracking.

8 LLM Disclosure

Here is a series of prompts (or the kind of prompts) given to LLMs, to help us assist during the course of this project.

1. Makefile fixes (with code snippets)
This prompt was given to generate the correct makefile which aligns with the user's C++ compiler and Raylib location.
2. UI offset fixes (with code snippet)
This prompt was given to fix an alignment issue with the game window and the grid. It resolved the issue where the padding and grid offset parameters were used interchangeably.
3. LaTeX pseudocode formatting (pseudocode given)
This prompt was given to convert the pseudocode entered by the user to make it LaTeX-style using the `algpseudocode` and `algorithmic` packages.
4. If a color does not get a solution path inside its segmented region, what should be done? should we expand the region or discard the segmentation?
5. What if connecting one color first blocks another color that had a valid path earlier?
6. If two colors have the same number of candidate paths, how do we decide which one to connect first?
7. As discussed earlier, the naive approach of divide and conquer by splitting up into 4 different quarters is kinda messy.
Gemini's response was, "Hey why dont you fix 8 corners of entry and exit for your path using locally using a network flow algorithm".
I doubt I understood anything. But what i did take back was the very idea of fixing the terminals which cross over first, and then worry about the rest.

To also make my life easier, I adapted the bisection strategy.
The initial merge-sort like algorithm (where you divide solve locally, combine by joining straddling terminals) to quick-sort like algorithm (where you first join straddling terminals and solve the halves recursively)
8. How to find perpendicular points to a line?
9. How to check if a point is inside a polygon?
10. Region finding cost evaluation
There was a bit of confusion on how the weighted Dijkstra would work by choosing the appropriate region. I was suggested to apply a smaller

(negative) cost for larger regions so that it favours the shortest path.

11. Debug flag addition

I required some print statements to evaluate the region finding algorithm efficacy and to check if it is being called correctly in the main algorithm function.